Diffusion Models: From Adding Noise to Generating Images
A Detailed Study of Latent Diffusion and Conditional Control

[Full name 1] – [Student ID 1] [Full name 2] – [Student ID 2]
[Full name 3] – [Student ID 3]
University of Science (HCMUS), VNU-HCM
CS420 – Artificial Intelligence

Abstract

Diffusion models generate images by learning to reverse a gradual corruption process. This report connects that intuition to Gaussian forward transitions, noise-prediction training, and iterative sampling. A scalar calculation and an original five-level image sequence make the mechanism concrete. The main focus is latent diffusion: a variational autoencoder compresses images, a conditional denoiser models the compressed representation, and a decoder returns it to pixels. Cross-attention and classifier-free guidance explain how prompts influence generation. A published pixel-versus-latent inpainting comparison supports a conditional efficiency advantage, while our analysis separates spatial compression from faster sampling. Reconstruction error, imperfect semantic control, data dependence, bias, and copyright concerns limit what efficiency and visual realism alone establish.

1. Introduction

How can learning to remove noise enable a model to generate new images? A denoiser trained on many images learns which structures are plausible at different noise levels. Starting from random noise, repeated learned updates can therefore produce a new sample from an approximation to the training distribution. It need not recover a particular image that was previously corrupted.

Pixel-space diffusion makes these updates over an entire image. Latent diffusion instead separates *representation learning* from *generative modeling*. The guiding question of this report is: *when does compression reduce the cost of diffusion while retaining useful visual detail and conditional control?* We review DDPM [6] and LDM [10], then connect their mechanisms to an evidence-supported comparison.

Our scope is explanatory and analytical. We compute a forward process locally; we do not train a denoiser or claim to have run a pretrained image generator. Results from the literature are explicitly identified in Section 5.

2. Background and Paper Review

2.1. Generative versus discriminative models

A discriminative model estimates a label from an observation, for example $p_\omega(y \mid x)$ for classifying an image x as a cat. Here y is a label and ω denotes learned parameters. A generative model represents a distribution such as $p_\theta(x)$ or $p_\theta(x \mid y)$, allowing it to sample images, possibly conditioned on a label or text. A classifier's confidence alone does not provide an image sampling procedure.

2.2. DDPM: forward corruption and learned reversal

Intuitively, the forward process slowly replaces structure with Gaussian noise. The reverse model learns plausible transitions in the opposite direction; it is not given the actual noise used to corrupt a test image. For a vectorized image with D scalar coordinates, DDPM specifies [6, Sec. 2]

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t \mathbf{I}\Big), \tag{1}$$

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}), \tag{2}$$

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{j=1}^{t} \alpha_j, \quad \bar{\alpha}_0 = 1.$$

x_0 is a clean image sampled from the empirical data distribution p_{data}; $x_t \in \mathbb{R}^D$ is its noisy version. The integer $t \in \{1, \ldots, T\}$ indexes noise level, not seconds. $\beta_t \in (0, 1)$ is a dimensionless per-step variance; $\mathbf{I}$ is the D-dimensional identity matrix. The *schedule* is the sequence $(\beta_1, \ldots, \beta_T)$, which controls how quickly signal disappears. Equation (2) samples a marginal directly, using one fresh standard-normal vector ε; it need not simulate every preceding step.

The noise predictor $\varepsilon_\theta(x_t, t) \in \mathbb{R}^D$, with trainable parameters θ, minimizes the required simplified objective [6, Eq. 14]:

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\right]. \tag{3}$$

The expectation averages uniformly sampled timesteps, training images, and independent Gaussian noise; $\|\cdot\|_2^2$ sums

Table 1. Qualitative comparison of representative image generators. Synthesis from [3, 6, 14]; these are architectural tendencies, not universal performance rankings.

Family	Learning idea	Sampling	Main trade-off
GAN	Generator competes with a discriminator	Typically one generator pass	Fast synthesis; balancing the adversarial game can be difficult
Autoregressive	Predict each variable from earlier variables	Sequential pixel/token prediction	Explicit factorization; sequential generation can be costly
Diffusion	Predict corruption across noise levels	Repeated denoising of a whole state	Flexible conditioning; multiple network evaluations

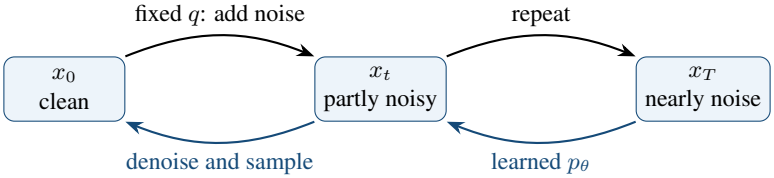


Figure 1. Original schematic of DDPM's two directions, based on [6]. Reverse arrows represent a learned distribution, not exact inversion of an individual forward trajectory.

squared coordinate errors. Training predicts the *aggregate* noise in Eq. (2). An epsilon-parameterized DDPM sampling update is

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) + \sigma_t u. \quad (4)$$

Here $u \sim \mathcal{N}(0, \mathbf{I})$ and $\sigma_t \geq 0$ is the chosen reverse standard deviation (e.g., $\sigma_t^2 = \beta_t$ for $t > 1$); the final step uses no added noise. Sampling starts at $x_T \sim \mathcal{N}(0, \mathbf{I})$ when $\bar{\alpha}_T$ is sufficiently small. DDPM's key contribution is effective noise-prediction training and its connection to denoising score matching, not the invention of all diffusion-based generative models.

3. Method and Worked Example

3.1. A complete one-dimensional calculation

For one step $t = 1$, choose normalized $x_0 = 0.8$, variance $\beta_1 = 0.1$, and Gaussian realization $\varepsilon = -0.5$. This larger variance makes the arithmetic visible; the image schedule below uses smaller steps. Equation (1) gives

$$\begin{aligned} \alpha_1 &= 1 - 0.1 = 0.9, \\ \sqrt{\alpha_1}\, x_0 &= \sqrt{0.9}\,(0.8) = 0.758947, \\ \sqrt{\beta_1}\, \varepsilon &= \sqrt{0.1}\,(-0.5) = -0.158114, \\ x_1 &= 0.758947 - 0.158114 = \boxed{0.600833}. \end{aligned}$$

Thus $q(x_1 \mid x_0 = 0.8) = \mathcal{N}(0.758947, 0.1)$: its mean is 0.758947, variance 0.1, and standard deviation $\sqrt{0.1} = 0.316228$. Across independent draws, $\mathbb{E}[x_1 \mid x_0] = \sqrt{0.9}\, x_0$ since $\mathbb{E}[\varepsilon] = 0$. The realization 0.600833 differs from that expectation; the conditional variance is β_1.

3.2. Five noise levels on an original small image

Our original 28×28 geometric house has foreground $+1$ and background -1, without an external dataset. The script `code/make_figures.py` simulates one Markov trajectory using Eq. (1) with independent noise at every step. It uses NumPy's default generator, seed 420, $T = 1000$, and the linear schedule $\beta_t = 10^{-4} + (t-1)(0.02 - 10^{-4})/999$, following DDPM's schedule choice [6, Sec. 4].

The last column is a schedule-based signal-to-noise coefficient ratio, also a per-coordinate variance ratio for unit-variance data. This illustrates corruption, not learned denoising or generation.

4. Latent Diffusion: Representation, Generation, and Control

4.1. Intuition: learn a useful compressed image space

An image contains both perceptually important structure and costly pixel detail. LDM first learns an image representation, then models its distribution with diffusion [10, Secs. 3.1–3.2]. The encoder is a learned analysis transform; it does not produce a smaller JPEG or a text description. Its spatial tensor retains features that a decoder can turn back into a plausible image.

For an RGB image $x_0 \in \mathbb{R}^{H \times W \times 3}$, a VAE encoder with parameters ϕ predicts an approximate posterior [8]:

$$q_\phi(\tilde{z} \mid x_0) = \mathcal{N}\big(\mu_\phi(x_0), \operatorname{diag}(\sigma_\phi^2(x_0))\big), \quad (5)$$

$$\tilde{z} = \mu_\phi(x_0) + \sigma_\phi(x_0) \odot \xi, \quad \xi \sim \mathcal{N}(0, \mathbf{I}),$$

$$\hat{x}_0 = D_\psi(\tilde{z}). \quad (6)$$



Figure 2. Original forward-noising illustration at five timesteps. The same trajectory is shown throughout. Values are clipped to $[-1, 1]$ only for display; stored arrays and calculations remain unclipped. The equation and schedule follow [6].

Table 2. Coefficients computed by the supplied script. Signal and noise columns refer to Eq. (2), not measured image quality.

t	$\bar{\alpha}_t$	Signal $\sqrt{\bar{\alpha}_t}$	Noise SD $\sqrt{1-\bar{\alpha}_t}$	$\bar{\alpha}_t/(1-\bar{\alpha}_t)$
0	1.000000	1.000000	0.000000	∞
50	0.971016	0.985401	0.170248	33.5015
200	0.659039	0.811812	0.583919	1.9329
500	0.078587	0.280334	0.959902	0.0853
1000	0.000040	0.006353	0.999980	0.000040

μ_ϕ and $\sigma_\phi > 0$ are elementwise latent means and standard deviations; diag forms a diagonal covariance, and $\odot$ is elementwise multiplication. ξ is *encoder* noise, distinct from diffusion noise ε. D_ψ is the decoder with parameters ψ; $\hat{x}_0$ is a reconstruction, not necessarily the original x_0. Reparameterization moves randomness into ξ, allowing gradients to pass through the encoder's outputs.

The tensor $\tilde{z} \in \mathbb{R}^{h\times w\times c}$ has $h = H/f$, $w = W/f$, latent channels c, and spatial downsampling factor f. For example, $256 \times 256 \times 3$ becomes $64 \times 64 \times 3$ for $f = 4, c = 3$, a configuration in LDM's autoencoder study [10, Table 8]. The number of spatial positions falls by $f^2 = 16$; channel count must also be considered when comparing storage.

What the autoencoder objective preserves

For a textbook VAE, minimizing the negative evidence lower bound gives [8]

$$\begin{aligned}\mathcal{J}_{\mathrm{VAE}} &= \mathbb{E}_{q_\phi(\tilde{z}|x_0)}[-\log p_\psi(x_0 \mid \tilde{z})] \\ &\quad + \mathrm{D}_{\mathrm{KL}}(q_\phi(\tilde{z} \mid x_0) \,\|\, \mathcal{N}(0, \mathbf{I}))\,.\end{aligned} \tag{7}$$

$p_\psi(x_0 \mid \tilde{z})$ is the decoder likelihood; its first term penalizes poor reconstruction. D_{KL} is the nonnegative Kullback–Leibler divergence in nats, encouraging a regular latent space. LDM's *KL-regularized* autoencoder is VAE-like but uses perceptual and patch-adversarial reconstruction objectives with a weak KL penalty [10, Sec. 3.1], rather than only a textbook pixel likelihood. The perceptual term compares learned visual features; the adversarial term encourages locally realistic texture. These can preserve perceptual quality without guaranteeing exact pixel recovery.

The original LDM paper also studies *VQ-regularized* autoencoders. Consequently, LDM is broader than VAE plus diffusion. We focus on the KL/VAE variant to satisfy Requirement 5. Crucially, the diffusion prior learns the encoded data distribution: weak KL regularization does not make that distribution exactly standard normal.

4.2. Diffusion in latent space and the complete pipeline

After autoencoder training, its parameters are fixed for diffusion training. Let $z_0 = a\tilde{z}$, where $a > 0$ is an optional fixed scale chosen to keep latent magnitudes suitable for the noise schedule. Take $a = 1$ when no rescaling is used; decoding must undo the same scale. The Gaussian construction of Section 2.2 now acts on $d_z = hwc$ latent coordinates, not RGB pixels:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon_z, \quad \varepsilon_z \sim \mathcal{N}(0, \mathbf{I}_{d_z}), \tag{8}$$

$$L_{\mathrm{LDM}} = \mathbb{E}_{x_0,y,t,\xi,\varepsilon_z}\left[\|\varepsilon_z - \varepsilon_\theta(z_t, t, \tau_\eta(y))\|_2^2\right]. \tag{9}$$

y is the condition (here a prompt), and τ_η is its encoder with parameters η. ε_θ is a time-conditioned U-Net that returns a tensor of the same shape as z_t. Its down/up paths and skip connections combine context with spatial detail. Equation (9) is the conditional noise-prediction objective of LDM [10, Eq. 3], written to show stochastic VAE encoding explicitly. A deterministic encoder or posterior-mean encoding omits ξ.

For generation, replace x by z in Eq. (4) and use the conditional predictor, or use a compatible sampler such as

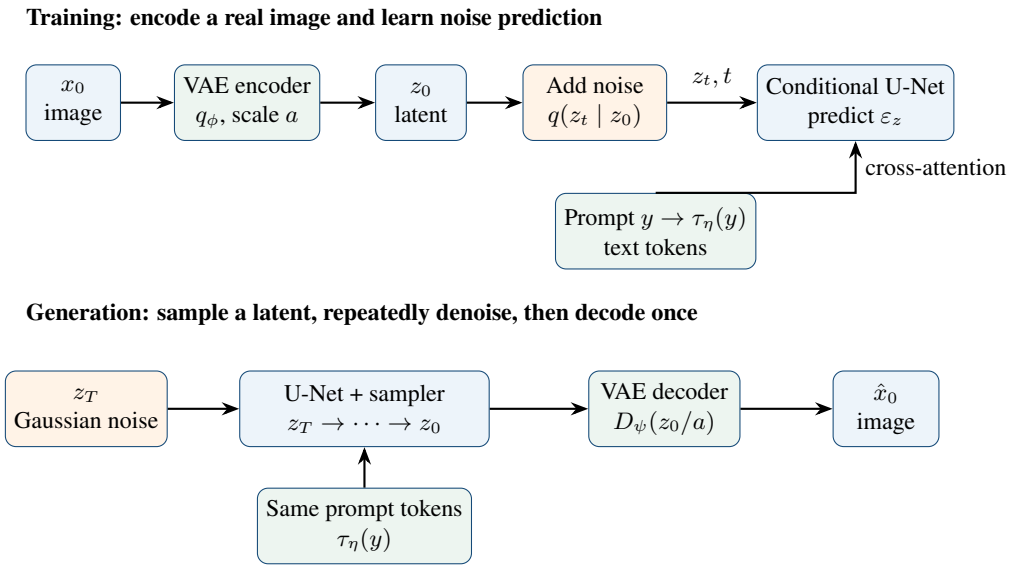


Figure 3. Original latent diffusion pipeline, based on [10, Fig. 3 and Sec. 3]. The VAE is learned first and frozen for diffusion training; its decoder runs after the denoising loop. Pure text-to-image generation does not require a real input image or the image encoder.

DDIM [12]. Starting with latent Gaussian noise, the sampler produces an approximate draw from the learned $p_\theta(z_0 \mid y)$; the final image is $\hat{x}_0 = D_\psi(z_0/a)$. This is why the decoder is not a diffusion denoiser: it maps a completed latent into pixels, while the U-Net repeatedly predicts noise.

One training update. Encode a paired image and prompt; sample a uniform timestep and Gaussian latent noise; construct z_t; predict that noise and backpropagate Eq. (9) into the denoiser (and condition encoder when trainable). No full reverse chain is needed for this update. At inference, parameters stay fixed while the sampler repeatedly updates the latent state. This distinction separates learning from generation.

4.3. Cross-attention: how language reaches the denoiser

The prompt does not directly overwrite pixels. Instead, every spatial feature can retrieve relevant information from prompt tokens. For a prompt such as *a red cube beside a blue sphere*, the text encoder produces contextual token embeddings; the denoiser learns which embeddings help its current spatial features. This interaction can occur at several U-Net resolutions and timesteps [10, Sec. 3.3].

Use a *row-vector* convention. At one U-Net layer, flatten its feature map into $F \in \mathbb{R}^{n \times d_u}$, where n is the number of spatial positions and d_u is its channel width. Let $C = \tau_\eta(y) \in \mathbb{R}^{m \times d_y}$ contain m token embeddings of width d_y.

One cross-attention head computes [10, 15]

$$Q = FW_Q, \quad K = CW_K, \quad V = CW_V, \tag{10}$$

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_a}}\right), \qquad O = AV. \tag{11}$$

The learned projections have shapes $W_Q \in \mathbb{R}^{d_u \times d_a}$, $W_K \in \mathbb{R}^{d_y \times d_a}$, and $W_V \in \mathbb{R}^{d_y \times d_v}$. Thus $Q \in \mathbb{R}^{n \times d_a}$ contains *queries from image features*, $K \in \mathbb{R}^{m \times d_a}$ contains *keys from text*, and $V \in \mathbb{R}^{m \times d_v}$ contains text values. d_a, d_v are positive integer head widths. A superscript $\top$ denotes transposition. Softmax acts across the m tokens for each spatial row, making $A \in \mathbb{R}^{n \times m}$ nonnegative with row sums of one. $O \in \mathbb{R}^{n \times d_v}$ supplies text-dependent features that are projected back into the U-Net's residual stream.

A numerical attention example

Take one spatial query $Q = (1, 0)$, two keys $K_1 = (\sqrt{2}, 0)$, $K_2 = (0, 0)$, and $d_a = d_v = 2$. The scaled logits are $(1, 0)$, so $A = (e/(e+1), 1/(e+1)) \approx (0.7311, 0.2689)$, where e is Euler's number. For $V_1 = (1, 0)$ and $V_2 = (0, 1)$, the output is $O \approx (0.7311, 0.2689)$. This original toy example shows weighted retrieval, not an observed attention map from an actual model.

Cross-attention differs from self-attention because queries and keys come from different representations. It also differs from concatenating an image condition channel-wise. Its weights express learned associations, not a guaranteed object mask or proof that a prompt relation has been satisfied.

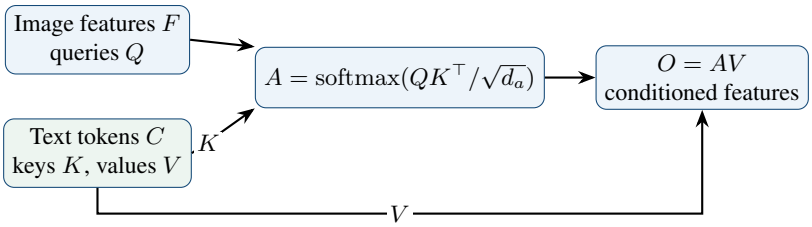


Figure 4. Original single-head cross-attention schematic, based on [10, 15]. Multi-head attention learns several such projections and combines their outputs.

4.4. Prompt control and classifier-free guidance

Cross-attention supplies the condition; *classifier-free guidance* (CFG) adjusts how strongly a conditional prediction influences sampling [5]. Train the same network with the condition replaced by a null condition $\varnothing$ with probability $p_{\text{drop}} \in (0, 1)$. At inference define

$$\begin{aligned} \varepsilon_u &= \varepsilon_\theta(z_t, t, \tau_\eta(\varnothing)), \\ \varepsilon_c &= \varepsilon_\theta(z_t, t, \tau_\eta(y)), \\ \varepsilon_{\text{cfg}} &= \varepsilon_u + s(\varepsilon_c - \varepsilon_u). \end{aligned} \tag{12}$$

$s \geq 0$ is a dimensionless guidance scale. Here $s = 0$ uses the unconditional prediction, $s = 1$ gives ordinary conditional sampling, and $s > 1$ extrapolates in the conditional direction. The CFG paper writes the equivalent expression $(1 + w)\varepsilon_c - w\varepsilon_u$; its parameter is $w = s - 1$. Stating the convention prevents confusing two scales with different numerical baselines.

For an original scalar illustration, $\varepsilon_u = 0.2$, $\varepsilon_c = 0.1$ and $s = 3$ give $\varepsilon_{\text{cfg}} = -0.1$. Guidance is therefore an extrapolation, not a convex average. Substituting it into the reverse update changes the generation trajectory. Greater guidance can improve conditional fidelity while reducing diversity; it does not ensure monotonically better quality [5]. Standard CFG requires two denoiser evaluations per step, often batched together, adding compute and potentially peak memory.

Practical control. To inspect a prompt change, hold the checkpoint, initial noise seed, sampler, step count, resolution, and s fixed. Change one attribute or relation at a time, such as *red cube* to *green cube*. Evaluate whether the specified attribute changed and whether unrelated content remained stable. A fixed seed improves comparability, but changing text can still reorganize the entire image. Counting, spatial relations, and binding attributes to the correct objects require explicit checks; a plausible-looking result is insufficient. Appendix B gives a future protocol, not claimed prompt results.

4.5. LDM paper: method, contributions, and differences

Rombach et al. [10] make three connected methodological contributions relevant here. First, a reusable perceptual autoencoder separates image compression from training a generative prior. Second, diffusion on a moderately compressed spatial grid reduces repeated high-resolution computation. Third, cross-attention provides a general interface for conditions such as text or layouts. Their compression study also shows why more compression is not automatically better: overly small grids discard detail that the generative model cannot faithfully recover.

Compared with DDPM, LDM changes the *representation* on which the noise-prediction objective operates and introduces an autoencoding boundary; it retains the central denoising principle. Compared with earlier autoregressive latent generators, it uses iterative spatial denoising rather than a chosen sequential token ordering. Compared with DDIM, it addresses the cost of each state evaluation; DDIM addresses the sampling trajectory and number of evaluations. These ideas can be combined. The original LDM paper should also be distinguished from any particular later Stable Diffusion checkpoint, dataset, text encoder, or user interface.

5. Results and Discussion

5.1. Evidence provenance and observed corruption

This section combines two kinds of evidence: **original calculations** from Section 3, and **published model results** below. No denoising, text-to-image, GPU latency, or peak-memory benchmark was executed for this report.

Figure 2 shows the house becoming difficult to recognize as the signal multiplier falls from 1 to 0.00635. Table 2 explains that behavior through the schedule rather than visual inspection alone. A single trajectory does not establish a population-level quality trend, and display clipping can hide changes in extreme values. These results validate the forward construction; the learned reverse process requires independent model evidence.

5.2. A published pixel-versus-latent setting

We select the *Places inpainting* comparison in Rombach et al. [10, Sec. 4.5, Table 6]: pixel-space LDM-1 versus LDM-4 with a KL-regularized first stage and attention. Their denoisers have a matched parameter count. The published protocol trains on 256×256 crops; the supplementary protocol specifies fixed validation masks and a 2,000-image validation set. We preserve the table's two sampling resolutions and six-epoch endpoint.

Our arithmetic gives sampling-throughput ratios $0.97/0.26 = 3.73$ at 256^2 and $0.34/0.07 = 4.86$ at 512^2. The epoch-time ratio is $20.66/7.66 = 2.70$ and training-throughput ratio $0.32/0.11 = 2.91$. The reported FID is 9.53 points lower, a relative decrease of 38.5%; this percentage is *not* a percentage increase in perceptual quality.

FID (Fréchet Inception Distance) compares feature-distribution statistics between generated and reference images [4]. It does not directly measure prompt adherence, fairness, factual correctness, or the quality of every individual image. The result supports an efficiency–quality advantage for this configuration and training endpoint. Six equal epochs do not imply equal wall-clock training budgets or equal convergence. Moreover, the table flags GPU-setting/batch-size differences from the paper's Figure 7; its ratios should not be universalized. Separate autoencoder pretraining is an additional cost not isolated by this table. Inpainting uses spatial conditions, so this experiment does not itself establish a text-prompt-control advantage.

5.3. Speed, quality, memory, and control: explicit criteria

Table 4 connects the benchmark to the mechanisms developed in Section 4. Its memory and control entries are analytical comparisons, not unreported empirical measurements.

Where speed comes from. An analytical latency decomposition is

$$\mathcal{T}_{\text{gen}} \approx \mathcal{T}_{\text{text}} + K\, r\, \mathcal{T}_{\text{U-Net}} + \mathcal{T}_{\text{decode}}, \tag{13}$$

where every $\mathcal{T}$ is measured in seconds, K is the number of sampling updates, and r is the effective number of denoiser evaluations per update (1 without standard CFG, 2 with it, before batching effects). The final term is absent in pixel diffusion. Latent compression mainly reduces the cost of a U-Net evaluation; DDIM can reduce K [12]. Fewer updates, higher guidance, and greater compression are separate design choices with different quality consequences.

Why a smaller state is not a guaranteed memory ratio. At 256^2, RGB has $196{,}608$ coordinates; the illustrative $64 \times 64 \times 3$ latent has $12{,}288$, giving a 16:1 state-element ratio. Generally it is $3f^2/c$. At fixed feature width, spatial activation storage also benefits from fewer positions, but model weights, attention, batching, and the full-resolution decoder affect the peak. Therefore neither a $16\times$ runtime speedup nor a $16\times$ total-memory reduction follows from this arithmetic.

Quality and applicability. Moderate compression can make iterative generation practical at larger resolutions. Tasks needing exact text, tiny structures, or pixel-faithful reconstruction must also inspect autoencoder error. Stronger guidance and lower FID cannot substitute for checking requested object attributes and relations. These distinctions explain why the favorable inpainting result is useful evidence without settling every application.

5.4. Training data, bias, copyright, and limitations

Data dependence. The VAE learns what can be represented, the denoiser learns what is likely, and the text encoder learns which language features connect to imagery. Errors or underrepresentation at any stage can propagate to outputs. Large image–text corpora such as LAION-5B illustrate the scale of this dependency [11]; they are not interchangeable with the specific datasets used in every original LDM experiment. Caption noise, duplicate images, language imbalance, and missing provenance are distinct concerns. Our analytical recommendation is to document dataset versions, sources, filters, and evaluation splits, and audit duplicates and underrepresented concepts.

Bias and prompt control. Bianchi et al. [1] report demographic stereotype amplification in text-to-image systems, including prompts that do not explicitly name demographic groups. Latent compression does not remove these associations. Because guidance amplifies conditional preferences and can reduce diversity, stronger prompt adherence should not be assumed to imply fairer representation. Evaluation should compare distributions across multiple prompts, groups, and seeds; prompt rewriting alone is insufficient evidence of mitigation.

Memorization and copyright. Carlini et al. [2] demonstrate extraction of memorized training examples from diffusion models. This establishes a possible failure mode, not that every output is copied. Dataset availability, permission to use underlying works, memorization risk, and rights in an output are separate questions. The U.S. Copyright Office's 2025 pre-publication analysis distinguishes data collection, training, outputs, and licensing [13]; it is a U.S. policy source, not Vietnamese law. Compression is no technical guarantee against reproduction. Our recommendations are documented provenance, appropriate permissions, deduplication, and similarity checks for sensitive use cases; none alone certifies legal compliance.

Limits of this report. Our original evidence covers a scalar calculation and one synthetic forward trajectory. The model comparison is a published inpainting experiment, not our own matched text-to-image benchmark. We have no measured peak-memory values or prompt-adherence scores.

Table 3. Published inpainting comparison, transcribed from [10, Table 6]. These are the paper's measurements, not a benchmark run by our team. Higher throughput and lower FID are better.

Criterion	Pixel LDM-1	LDM-4 (KL, with attention)
Training throughput (samples/s)	0.11	0.32
Sampling throughput at 256^2 (samples/s)	0.26	0.97
Sampling throughput at 512^2 (samples/s)	0.07	0.34
Training + validation (hours/epoch)	20.66	7.66
Validation FID after epoch 6 (2k samples)	24.74	15.21
Peak memory (GB)	Not reported in this table	

Table 4. Pixel versus latent diffusion. The concrete latent setting is LDM-4 with KL regularization. Architecture-based analysis is informed by [5, 6, 10]; measured values are confined to Table 3.

Criterion	Pixel diffusion	Latent diffusion (KL, $f = 4$)
Speed	Repeatedly processes a pixel-resolution state	Smaller repeated states; one final decode. The published inpainting throughput is higher in the selected setting
Quality	No separate autoencoder bottleneck; still limited by denoiser, data, and sampling	Compression can retain useful structure but lose detail or introduce reconstruction artifacts
Memory	Larger spatial states and potentially larger feature maps	Fewer latent positions; autoencoder/text-encoder weights and decoder activations add memory. End-to-end peak memory needs measurement
Control	Can support text conditioning and guidance when trained for it	Cross-attention and guidance support flexible prompts; compression itself does not guarantee better semantic control

A future study should hold hardware, precision, batch size, sampler, step count, guidance, resolution, and dataset fixed, report both autoencoder and denoiser costs, and evaluate quality and control separately.

6. Conclusion

Learning noise removal across corruption levels supplies a generative sampling rule. Latent diffusion retains this principle while moving its repeated computation into a learned spatial representation. The VAE determines the balance between compression and reconstruction; the denoiser learns the latent distribution; cross-attention and guidance connect prompts to sampling. Published inpainting evidence supports a substantial efficiency benefit for a concrete KL-regularized LDM-4 setting, with lower FID at the reported endpoint. The broader conclusion is conditional: useful compression can improve practicality, while reconstruction fidelity, semantic control, data quality, bias, and rights concerns still require their own evaluation.

References

[1] Federico Bianchi, Pratyusha Kalluri, Esin Durmus, Faisal Ladhak, Myra Cheng, Debora Nozza, Tatsunori Hashimoto, Dan Jurafsky, James Zou, and Aylin Caliskan. Easily accessible text-to-image generation amplifies demographic stereotypes at large scale. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*, 2023.

[2] Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwag, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. Extracting training data from diffusion models. In *32nd USENIX Security Symposium*, 2023.

[3] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.

[4] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.

[5] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022. A short version appeared at a NeurIPS 2021 workshop.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020.

[7] IEEE. ICCV 2025 Author Kit, 2025. Official author kit; style and bibliography files distributed with the project.

[8] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.

[9] OpenAI. Codex, 2026. AI assistance for drafting, LaTeX, code, and source checking; see Appendix B.

[10] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. arXiv v2, April 2022; includes supplementary material.

[11] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, et al. LAION-5B: An open large-scale dataset for training next generation image-text models. In *Advances in Neural Information Processing Systems*, 2022.

[12] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

[13] U.S. Copyright Office. Copyright and artificial intelligence, part 3: Generative AI training. Technical report, U.S. Copyright Office, 2025. Pre-publication version; consulted November 5, 2026. A U.S. policy analysis, not a statement of Vietnamese law.

[14] Aäron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1747–1756, 2016.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

A. Requirement-to-Evidence Map

This map concerns the written report. The assignment's separate slide, oral-presentation, balanced-participation, and Q&A requirements require presentation deliverables and cannot be fulfilled by a report alone. The main text occupies pages 1–7; references and appendices are separate.

Table 5. Coverage of the supplied CS420 S06 seminar brief.

Task	Required content	Location / evidence
1	Generative versus discriminative; GAN, autoregressive, diffusion	Section 2.1; Table 1
2	Forward process, beta schedule, Gaussian noise, noise predictor, reverse sampling; both-direction diagram	Section 2.2; Eqs. (1)–(4); Figure 1
3	Manual 1D forward step, symbols, stochastic expectation	Section 3.1; $x_1 = 0.600833$; conditional mean and variance
4	Small image at five noise levels; optional pretrained demonstration	Section 3.2; Figure 2; Table 2; executable code and arrays. Pretrained generation was not performed
5	VAE encoder/decoder, latent operations, cross-attention, pipeline	Sections 4.1–4.3; Eqs. (6)–(11); Figures 3–4
6	Speed, quality, prompt control, data, copyright, bias; explicit pixel/latent comparison	Sections 4.4, 5.2–5.4; Tables 3–4

Required representations. Equation (1) is the Gaussian forward transition; Eq. (3) is the simplified noise-prediction objective. The worked example gives a complete substitution, result, and interpretation. Appendix C records ranges, units, and modeling assumptions.

Paper reading and synthesis. DDPM's forward/reverse method and contribution are reviewed in Section 2.2. LDM's representation, method, conditioning, contributions, and differences are developed in Sections 4–4.5. Section 5.2 identifies the exact source table for performance claims. The report includes Abstract, Introduction, Background/Paper Review, Method/Worked Example, Results and Discussion, Conclusion, and References. English is used according to the commissioning instruction, which overrides the brief's Vietnamese-language request.

B. Reproducibility, Evaluation Protocol, and Attribution

B.1. Reproduce the original illustration

The Overleaf project already contains the generated figure; Python is not needed to compile the report. To regenerate it locally:

```
python -m pip install -r code/requirements.txt
python code/make_figures.py
latexmk -pdf -interaction=nonstopmode -halt-on-
    error main.tex
```

The figure uses a synthetic geometric image, not MNIST or another dataset. The script records the seed, schedule coefficients, library versions, scalar example, and benchmark-ratio arithmetic in `figures/analysis.json`. It saves unclipped frames, timesteps, the schedule, and cumulative coefficients in `noise_arrays.npz`, and summary coefficients in `noise_levels.csv`. The delivered run used NumPy 2.3.5 and ReportLab 4.4.9. The sequence is one sampled Markov path; identical seeds and library versions reproduce it. No image-generation checkpoint is involved.

B.2. A future prompt and runtime study (not executed)

Select and record exact checkpoint identifiers, revisions, dataset licenses, hardware, software versions, numerical precision, resolution, and batch size. Keep each fixed within a comparison and record warm-up policy. For latent models, measure text encoding, denoising, decoding, and end-to-end latency separately, and record peak allocated memory over the entire pipeline.

Use paired prompts such as *a red cube to the left of a blue sphere* and *a green cube to the left of a blue sphere*; separately swap the spatial relation. Predefine at least 20 seeds per condition and inspect all samples. Vary sampling steps and guidance in separate sweeps. Report object presence, color binding, relation correctness, and diversity; do not select only attractive examples. This is a proposed evaluation design, not empirical evidence of a model's prompt capability.

B.3. Provenance and AI assistance

Figures 1, 3, and 4 are original TikZ schematics explaining cited methods; no paper figure is reproduced. Figure 2 and Table 2 are generated by the supplied code. Table 3 is transcribed from LDM Table 6; ratios are our arithmetic. The two qualitative comparison tables are analytical syntheses with sources in their captions.

The document uses the requested ICCV 2025 author kit [7], with its official style and bibliography files. It uses 10-point type, two columns, and US Letter paper, following the new template request. The author block retains HCMUS and CS420 information and three name/ID placeholders. Template provenance is included in the project.

OpenAI Codex [9] assisted with drafting, equations, code, diagram construction, source checking, and LaTeX preparation. Preparation checks included consulting the original DDPM/LDM equations and LDM Table 6, recalculating the numerical examples and ratios, and compiling and visually reviewing the PDF. The named student authors should review and accept the final manuscript and replace the author placeholders before submission. No human study, new trained model, or pretrained generation benchmark is claimed.

C. Notation, Assumptions, and Additional Derivation

C.1. Ranges and units

Images are normalized and dimensionless. Clean pixels in the illustration lie in $[-1, 1]$; noisy states, latent values, and predicted noise are real-valued and need not stay within that interval. There is no physical time unit in the diffusion index.

Table 6. Notation used in the equations; meanings are also introduced beside the corresponding equations in the main text.

Symbols	Meaning, range, and units
$x_0, x_t, \hat{x}_0$	Clean/noisy/reconstructed image; vector in $\mathbb{R}^D$ or an equivalent image tensor; normalized intensity units
D, H, W, h, w, c, d_z	Positive integer dimensions. $D = 3HW$ for RGB, $d_z = hwc$; H, W count pixels, h, w count latent positions
t, T, j	Integer timestep, terminal step, and product index; $1 \le t \le T$, $1 \le j \le t$
$\beta_t, \alpha_t, \bar{\alpha}_t$	Dimensionless schedule coefficients; $0 < \beta_t < 1$, $\alpha_t = 1 - \beta_t$, $0 < \bar{\alpha}_t \le 1$
$q, p_\theta, p_{\text{data}}$	Fixed forward transition, learned reverse/generative distribution, and data distribution, respectively
$\mathcal{N}(\mu, \Sigma), \mathbf{I}$	Gaussian with mean μ and covariance Σ; identity covariance of the relevant dimension
$\varepsilon, \varepsilon_z, u, \xi$	Independent standard-normal draws in their respective sampling operations; each coordinate has mean 0 and variance 1
$\sigma_t, \mu_\phi, \sigma_\phi$	Reverse noise standard deviation $\sigma_t \ge 0$; VAE posterior mean and positive standard deviations
$\theta, \phi, \psi, \eta, \omega$	Trainable parameter arrays of denoiser, VAE encoder, decoder, condition encoder, and illustrative classifier
$\tilde{z}, z_0, z_t, f, a$	Raw/clean-scaled/noisy latent; spatial factor $f \ge 1$, with H, W divisible by f; fixed dimensionless scale $a > 0$
$y, \tau_\eta(y), F, C$	Condition, encoded condition, U-Net features, and token embedding matrix. Continuous embeddings are real-valued
n, m, d_u, d_y, d_a, d_v	Positive integer counts and widths for cross-attention; see Section 4.3 for all matrix shapes
Q, K, V, A, O	Query, key, value, attention-weight, and output matrices; $A_{ij} \in [0, 1]$ and $\sum_j A_{ij} = 1$
$s, w, p_{\text{drop}}, \varnothing$	CFG scale $s \ge 0$, alternative scale $w = s - 1$, dropout probability in $(0, 1)$, and null condition
$K, r, \mathcal{T}$	In the cost model only, K counts updates (not attention keys), r counts evaluations/update, and $\mathcal{T}$ is seconds
$L_{\text{simple}}, L_{\text{LDM}}$	Expected summed squared noise-prediction errors; dimensionless squared normalized-coordinate units
$\mathbb{E}, \|\cdot\|_2, \mathrm{D}_{\text{KL}}$	Expectation, Euclidean norm, and KL divergence; natural logarithms make KL and negative log likelihood use nats

C.2. Why the forward marginal is available in closed form

Assume independent standard-normal innovations $\varepsilon^{(1)}, \varepsilon^{(2)}$ and let x_0 be fixed. Applying Eq. (1) twice gives

$$x_1 = \sqrt{\alpha_1} x_0 + \sqrt{\beta_1}\varepsilon^{(1)},$$
$$x_2 = \sqrt{\alpha_2\alpha_1} x_0 + \sqrt{\alpha_2\beta_1}\varepsilon^{(1)} + \sqrt{\beta_2}\varepsilon^{(2)}.$$

The two independent noise terms have total variance $\alpha_2\beta_1 + \beta_2 = \alpha_2(1-\alpha_1) + (1-\alpha_2) = 1 - \alpha_2\alpha_1$. A sum of independent Gaussians is Gaussian. Repeating this argument yields Eq. (2), with mean $\sqrt{\bar{\alpha}_t} x_0$ and covariance $(1-\bar{\alpha}_t)\mathbf{I}$ [6].

Gaussian innovations are independent across coordinates and steps, but the noisy images themselves have correlations inherited from x_0. When $\bar{\alpha}_T$ is close to zero, dependence on the starting image is weak and a standard-normal starting prior is a useful approximation. A finite schedule need not make it exactly equal to that prior. The learned Gaussian reverse transition is a modeling approximation motivated by small forward steps; no finite network is assumed to know the exact reverse conditional.

C.3. Why noise prediction is useful without knowing the true noise

For a fixed x_t and t, the vector minimizing expected squared prediction error is $\mathbb{E}[\varepsilon \mid x_t, t]$ (and, in a conditional model, $\mathbb{E}[\varepsilon \mid x_t, t, y]$). Thus a trained predictor estimates a statistical direction shared by plausible clean examples, not an oracle that recovers every random draw. Its corresponding clean-state estimate is

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{14}$$

This follows by rearranging Eq. (2); prediction errors remain and can be magnified at small $\bar{\alpha}_t$. The reverse sampler uses predictions over many levels rather than treating one estimate as guaranteed recovery. The same argument applies in latent space, followed by the learned decoder.

C.4. Two training stages and their different roles

Stage one updates encoder ϕ and decoder ψ to preserve useful image information in a regularized representation. Stage two fixes them and trains denoiser θ with Eq. (9). The original LDM conditional formulation jointly optimizes the condition encoder and denoiser [10, Sec. 3.3]; particular later implementations may instead keep the text encoder fixed.

The distinction matters: improving the denoiser cannot guarantee exact recovery of details that the first stage systematically discards. Conversely, a high-quality autoencoder does not itself provide the conditional latent distribution needed for generation. The VAE prior regularizer and the learned diffusion prior play different roles.